

UNIVERSITATEA TRANSILVANIA DIN BRAȘOV
Facultatea de Inginerie Electrică și Știința Calculatoarelor
Specializarea Automatică și Informatică Aplicată

Caiet de practică

Practică de specialitate

Student: Jipa Ștefan Ciprian
Grupa: 4LF442
An de studiu: Anul II
Perioada practicii: Iunie – August 2026
Firma / locul de practică: Vanadium Systems SRL
Funcție în firmă: Fondator & Full-stack Developer

Document elaborat în locul caietului de practică standard, cu acordul
prodecanului responsabil de practică, conform cerințelor comunicate:
tehnologii de dezvoltare utilizate, proiectele realizate și fluxul de lucru al unui proiect.

Cuprins

Context	2
1 Tehnologii de dezvoltare folosite	2
1.1 Principii arhitecturale	2
1.2 Stack principal (aplicații SaaS)	2
1.3 Automatizări & produse orientate pe AI conversațional	3
1.4 Instrumente de dezvoltare asistată de AI	3
1.5 Securitate & conformitate	4
2 Arhitectura tehnică a unei platforme SaaS	4
2.1 Structura pe module	4
2.2 Convenții de bază de date	5
2.3 Convenții de denumire	5
3 Proiecte realizate	5
3.1 VanaGreen — platformă SaaS pentru operatori de reciclare (proiect principal) . .	6
3.2 VanaDash — hub intern de monitorizare (proiect secundar)	7
3.3 Alte produse SaaS live	7
3.4 Proiect în dezvoltare: platformă SaaS pentru service-uri auto	8
4 Workflow-ul unui proiect	9
4.1 Discuția cu omul din spatele nevoii	9
4.2 Planificare	9
4.3 Implementare pe etape (“wave”)	9
4.4 Testare	9
4.5 Revizuire de cod și securitate	10
4.6 Integrare continuă și livrare	10
4.7 Documentare	10
Concluzii	10

Context

Vanadium Systems SRL este o companie de software fondată și administrată de mine, care dezvoltă produse SaaS verticale și orizontale pentru piața din România. Fiind fondator unic (fără angajați), activitatea de practică a constat în dezvoltarea end-to-end a acestor produse: discuții cu operatorii din domeniu, arhitectură, backend, frontend, infrastructură, testare, securitate și mentenanță în producție: toate etapele unui produs software real, de la ideea inițială până la clientul care îl folosește zilnic.

Acest document acoperă, conform cerinței comunicate de prodecan în locul caietului standard: tehnologiile de dezvoltare folosite, proiectele realizate (fără nume de clienți) și fluxul de lucru urmat pentru un proiect. Am extins fiecare secțiune cu suficient detaliu tehnic cât să reflecte efectiv volumul de muncă din spatele ei, nu doar un rezumat de o linie pe subiect.

1. Tehnologii de dezvoltare folosite

1.1. Principii arhitecturale

Înainte de stack-ul concret, merită explicate două decizii arhitecturale care se regăsesc identic în toate platformele SaaS dezvoltate. Restul alegerilor tehnologice de mai jos sunt, în mare parte, consecințe ale acestor două.

Prima este multi-tenancy cu izolare completă a datelor. O singură instanță de aplicație (frontend și backend) deservește toți clienții, dar fiecare interogare către baza de date filtrează explicit după identificatorul de client, `tenant_id`. Fără excepții. E decizia cu cel mai mare impact din tot codul: o singură scăpare aici înseamnă date dintr-o firmă vizibile în alta, ceea ce e inacceptabil atât legal (GDPR) cât și contractual.

A doua este izolarea pe module de business. Fiecare capacitate funcțională (colectări, facturare, rapoarte etc.) e un modul separat, cu propriul cod de rutare, logică și schemă de date. Modulele nu importă logică unele din altele; când chiar trebuie să comunice, o fac explicit, prin baza de date comună sau printr-o magistrală internă de evenimente, niciodată prin apel direct. Motivul e practic: fiind un singur dezvoltator, am nevoie să pot modifica un modul fără să fiu obligat să țin minte cum funcționează tot restul aplicației în același timp.

1.2. Stack principal (aplicații SaaS)

Nivel	Tehnologii
Frontend	React 18, TypeScript, Vite, Tailwind CSS, shadcn/ui, TanStack Query, React Hook Form + Zod, Recharts, i18next
Backend	Node.js, Fastify, TypeScript, Drizzle ORM, PostgreSQL, node-cron, Puppeteer (generare PDF), MinIO (stocare fișiere), Nodemailer / Resend (e-mail)
Mobil	Capacitor (aceeași bază de cod React, aplicație Android/APK)
Infrastructură	Docker & Docker Compose, Nginx, Certbot (SSL), Hetzner VPS, Cloudflare (DNS, Pages, Functions, Turnstile)
Testare & CI/CD	Vitest, Playwright (E2E + accesibilitate), GitHub Actions (lint, type-check, test, build, deploy)

Câteva dintre aceste alegeri au o motivație destul de concretă, legată de faptul că sunt un singur dezvoltator care răspunde și de operare, nu doar de scris cod, așa că le detaliez puțin. Fastify e

ales împreună cu Zod pe fiecare rută, astfel încât validarea cererii și a răspunsului se face direct la nivel de framework, nu prin cod de validare scris manual pe care îl pot uita undeva. Drizzle ORM, în loc de SQL brut, generează tipuri TypeScript direct din schema bazei de date; dacă redenumesc o coloană, obțin erori de compilare imediat, în loc să descopăr problema abia la runtime, în producție.

Pentru infrastructură am ales Docker Compose pe server propriu (Hetzner, Germania) în locul unei platforme cloud gestionate, din două motive: cost fix lunar previzibil (contează pe un produs cu marjă mică per client) și rezidența datelor în UE, cerută explicit de clienții din România pentru conformitate GDPR. Iar pentru mobil folosesc Capacitor, ca să rulez aceeași bază de cod React și ca aplicație Android. Întreținerea a două codebase-uri separate, web și mobil nativ, pur și simplu nu e sustenabilă când lucrezi singur.

1.3. Automatizări & produse orientate pe AI conversațional

Categorie	Tehnologii
Orchestrare fluxuri	n8n, Make
Voce AI	Vapi
Mesagerie	WhatsApp Business API, Gmail API, Google Calendar API

Aceste produse nu sunt aplicații monolitice ca VanaGreen, ci fluxuri de automatizare construite peste API-uri oficiale (nu integrări neoficiale, fragile) și orchestrate prin n8n sau Make. Alegerea e motivată simplu, de viteză: un flux nou poate fi construit și pus în producție în câteva zile, nu săptămâni. Contează, pentru că e vorba de un produs orizontal, vândut la mai multe firme mici, fiecare cu cerințe ușor diferite de configurat.

1.4. Instrumente de dezvoltare asistată de AI

O parte semnificativă a fluxului de dezvoltare folosește un agent de programare AI (**Claude Code**, Anthropic) ca instrument principal de scriere și refactorizare a codului, sub supraveghere și revizuire umană directă la fiecare pas. Acest lucru este menționat separat pentru că schimbă efectiv fluxul de lucru descris în secțiunea 4: planificarea și specificarea cerințelor devin explicite și scrise (nu doar gândite), iar revizuirea de cod și verificarea automată a calității devin pași obligatorii înaintea oricărei livrări, nu opționali.

Concret, agentul este folosit pentru:

- implementarea de funcționalități noi pornind de la un plan scris în prealabil (structură DB, endpoint-uri backend, componente frontend);
- revizuire de cod automată (bug-uri, duplicare, simplificare) înainte de merge;
- scriere și menținere de teste automate;
- generare și actualizare de documentație tehnică internă (jurnal de decizii per director/modul).

Un plan scris (înainte de orice implementare) arată, redus la esențial, astfel:

Feature: prag de avertizare la colectari de la persoane fizice
 Scop: evitarea depasirii plafonului legal de 1000 lei / persoana / an
 fara ca operatorul sa observe manual suma cumulata

Schema DB: adaugare coloana 'cumulative_amount' (calculata) pe 'persons'
 Backend: validare in service-ul de creare colectare, inaintea de commit
 Frontend: warning inline in formular, cu optiune de split automat

Cazuri de test: sub prag / exact la prag / peste prag / split ulterior

Fiecare linie din acest plan devine, la implementare, un pas verificabil separat – diferența esențială față de a scrie cod direct, fără o etapă explicită de specificare.

1.5. Securitate & conformitate

Standardul de securitate e identic pe toate platformele SaaS, indiferent de dimensiunea clientului. La autentificare, JWT-ul stă exclusiv într-un cookie `HttpOnly`, niciodată în `localStorage`; MFA prin TOTP e obligatoriu pentru rolurile cu acces la date sensibile, un singur dispozitiv rămâne activ per utilizator, iar sesiunea expiră atât absolut cât și după inactivitate. La nivel de rută avem CSRF prin double-submit pe toate mutațiile, rate limiting (mai agresiv pe autentificare) și blocare temporară după încercări repetate eșuate de login.

Câmpurile cu date personale sunt criptate end-to-end: cheia se derivă cu PBKDF2 direct în browser, datele ajung pe server deja criptate cu AES-GCM, iar backend-ul nu le vede niciodată în clar. Toate acțiunile sensibile intră într-un jurnal de audit imuabil. La nivel de rețea, HTTPS e impus (HSTS activ), iar baza de date și storage-ul intern (MinIO) nu sunt expuse public — se accesează doar prin backend, cu autentificare. Backup-ul rulează automat, zilnic.

Secretele nu ajung niciodată în cod, în repository sau în variabile expuse către client; sunt gestionate exclusiv prin secretele platformei de hosting, iar fiecare rulare de integrare continuă scanează automat după secrete uitate accidental în cod. Pentru platformele cu date GDPR relevante, orice modificare care atinge câmpuri de date personale, generarea documentelor legale sau regulile de reținere/ștergere a datelor trece printr-o etapă de revizuire separată, dedicată strict acestor aspecte (detaliată în secțiunea 4.5) — nu se amestecă cu revizuirea tehnică obișnuită.

2. Arhitectura tehnică a unei platforme SaaS

Ca să nu rămână abstract ce înseamnă “modul izolat” menționat mai sus, arăt aici, la nivel de cod, cum sunt structurate concret platformele de tip VanaGreen sau VanaDash.

2.1. Structura pe module

Fiecare capacitate de business (colectări, facturare, rapoarte etc.) este un folder de sine stătător, cu exact trei fișiere responsabile de roluri diferite:

Fișier	Responsabilitate
<code>routes.ts</code>	Definește endpoint-urile HTTP, validarea Zod a cererii/răspunsului și rolurile necesare pentru acces. Nu conține logică de business.
<code>service.ts</code>	Logica efectivă: interogări către baza de date, reguli de validare, calcule. Este singurul loc care “știe” cum funcționează modulul respectiv.
<code>schema.ts</code>	Structura tabelor din acest modul, definită cu Drizzle ORM (sursă unică de adevăr, din care se generează atât tipurile TypeScript cât și migrațiile SQL).

Un exemplu simplificat (fără date reale), reprezentativ pentru tiparul folosit consecvent în toate modulele backend:

```
// routes.ts
fastify.patch('/:id', {
  preHandler: [authenticate, requireRole(['operator'])],
  schema: { body: UpdateBodySchema, response: { 200: ItemResponseSchema } },
}, async (req) => {
```

```

    return { success: true, data: await service.update(req.tenantId, req.params.id, req.body) }
  })

// service.ts -- actualizare cu izolare per client (tenant)
export async function update(tenantId: string, id: string, input: UpdateInput) {
  const [row] = await db.update(table)
    .set({ ...input, updatedAt: new Date() })
    .where(and(eq(table.id, id), eq(table.tenantId, tenantId), isNull(table.deletedAt)))
    .returning()
  if (!row) throw createError({ statusCode: 404, code: 'NOT_FOUND' })
  return row
}

```

Observație cheie: `tenantId` apare explicit în fiecare interogare care atinge date de business. Nu e opțional și nu e “adăugat mai târziu”. E convenția cu cea mai mare importanță din tot codul, pentru că o singură interogare fără acest filtru e suficientă pentru o scurgere de date între clienți.

2.2. Convenții de bază de date

Fiecare tabel de business respectă aceeași structură minimă, indiferent de proiect:

```

id          uuid primary key default gen_random_uuid()
tenant_id   uuid not null references tenants(id)
created_at  timestamp not null default now()
updated_at  timestamp not null default now()
deleted_at  timestamp nullable    -- stergere logica (soft-delete), niciodata hard-delete

```

Ștergerea definitivă a datelor de business (DELETE SQL) nu se folosește niciodată din cod. Orice “ștergere” setează doar `deleted_at`, păstrând istoricul recuperabil, necesar atât pentru audit intern cât și pentru anumite obligații legale de păstrare a evidențelor. Fiecare cheie străină are o politică explicită (`restrict`, `cascade` sau `set null`), stabilită deliberat, nu implicit.

2.3. Convenții de denumire

Consecvența denumirilor contează disproporționat de mult într-un cod întreținut de o singură persoană, pe termen lung. Reduce timpul de reorientare atunci când revin, peste luni de zile, la un modul pe care nu l-am mai atins.

Element	Convenție
Fișiere backend	kebab-case
Componente React	PascalCase
Hooks React	use-kebab-case
Variabile/funcții	camelCase, fără abrevieri neclare
Booleeni	prefix is/has/can (ex. <code>isLoading</code>)
Rute API	kebab-case, plural, fără verbe (ex. <code>/api/collections</code>)
Coloane DB	snake_case

3. Proiecte realizate

3.1. VanaGreen — platformă SaaS pentru operatori de reciclare (proiect principal)

Platformă SaaS multi-tenant care digitalizează complet activitatea unui operator de colectare deșeuri din România: înregistrarea colectărilor, gestiunea stocului, generarea automată a documentelor legale și rapoartele de conformitate. Este cel mai amplu și mai vechi produs al firmei, cel pe care s-a validat inițial arhitectura descrisă în secțiunea 3 și reutilizată apoi în celelalte proiecte.

3.1.1. Module funcționale

- **Colectări.** Flux digital complet de la sursă (companie, persoană fizică, instituție, campanie) până la stocare, cu avertismente automate pentru flux mixt de deșeuri pe aceeași colectare, depășirea plafonului legal de 1.000 lei pentru persoane fizice și posibile duplicate. Se pot despărți sau uni colectări ulterior, cu istoric complet al modificării.
- **Expedieri.** Stocul pleacă spre colectori sau procesatori doar după validarea automată a cantității disponibile — nu se poate expedia mai mult decât există fizic în stoc — iar documentele aferente se generează automat.
- **Tratare.** Deșeurile compuse se descompun în fracții valorificabile pe baza unor rețete predefinite per client (un frigider, de exemplu, iese ca fier, aluminiu, plastic și agent frigorific), cu stocul actualizat automat la finalizare.
- **Documente legale și rapoarte de conformitate.** NIR, proces-verbal de intrare și ieșire, borderou de plată cu calcul automat al reținerilor legale, anexe pentru deșeuri periculoase, toate cu numerotare configurabilă per client. La acestea se adaugă exporturi gata de depus pentru evidența deșeurilor, raportări de mediu și declarații fiscale legate de reținerile la sursă.
- **Facturare.** Emitere, stornare, urmărire plăți, regim de TVA detectat automat și integrare directă cu e-Factura ANAF (SPV).
- **Containere și flotă.** Istoricul mișcărilor (ridicare, predare, relocare) pentru containerele proprii și cele închiriate, plus alerte de mentenanță pentru ITP, asigurări, revizii și verificări metrologice ale cântarelor.
- **Aplicație mobilă.** Aplicație Android pentru echipele de teren, cu funcționare offline și sincronizare automată la revenirea conexiunii: colectări, fotografii, program de ridicări, toate introduse direct din teren.
- **API public și webhook-uri,** pentru integrare cu sisteme ERP/WMS pe care clientul le are deja, prin REST documentat și evenimente în timp real.
- **White-label:** logo, culori și domeniu propriu configurabile per client, ca aplicația să poarte brandul firmei client, nu al Vanadium Systems.

3.1.2. Securitate specifică domeniului

Pe lângă măsurile generale descrise în secțiunea 2.5, VanaGreen tratează separat câmpurile de date personale ale persoanelor fizice furnizoare de deșeuri (nume, CNP, date vehicul) – criptate end-to-end, niciodată vizibile în clar pe server.

3.1.3. Rezultate operaționale

Cifre reale, măsurate din utilizarea în producție a platformei:

Indicator	Valoare
Timp de la cântărire la document generat	sub 2 minute
Tipuri de documente legale generate automat	peste 15
Digitalizare a fluxului operațional	100% (fără hârtie)
Conformitate	GDPR (criptare, minimizare, drept la ștergere)

3.2. VanaDash — hub intern de monitorizare (proiect secundar)

Dashboard intern, cu un singur utilizator (fără clienți externi, fără multi-tenancy), construit pentru a agrega într-un singur loc datele din toate platformele Vanadium Systems, altfel imposibil de urmărit central de o singură persoană care rulează în paralel mai multe produse.

3.2.1. Ce face concret

La bază stau niște conectori per platformă: module TypeScript dedicate, fiecare știind să comunice cu API-ul unei singure platforme (de exemplu VanaGreen) și să extragă din ea metrice standardizate. Peste ei rulează job-uri programate (`node-cron`) care fac sincronizarea periodică și țin un jurnal de execuție pentru fiecare rulare: succes sau eșec, câte înregistrări, ce eroare, dacă a fost cazul.

Datele adunate așa alimentează indicatorii financiari și operaționali centralizați, cu filtrare pe perioade (luna curentă, ultimele 30 de zile, interval custom), plus un set de reguli de prag configurabile care declanșează alertare automată — de exemplu, venit sub un anumit nivel într-o zi. Singura excepție de la rolul strict de agregator este facturarea: facturile emise de firmă sunt create și numerotate direct aici, apoi transmise ca e-Factură către ANAF.

Tehnologii: aceeași bază tehnică ca VanaGreen (React, Fastify, PostgreSQL, Drizzle ORM), simplificată prin eliminarea completă a multi-tenancy-ului. E o decizie deliberată — o platformă cu un singur utilizator nu are de ce să care după ea complexitatea izolării per client.

3.3. Alte produse SaaS live

3.3.1. Asistent virtual AI (recepționist automat)

Preia apeluri telefonice, mesaje WhatsApp și e-mail simultan, non-stop, cu escaladare automată la un operator uman pentru cazurile complexe. E gândit pentru firme mici care nu își permit o recepție dedicată 24/7: clinici, cabinete, HoReCa, servicii B2B.

Vocea e personalizată pe tonul firmei și poate face programări direct în calendar, cu confirmare pe SMS sau WhatsApp, plus răspunsuri la întrebări frecvente pe care clientul le configurează singur, fără intervenție IT. Pe WhatsApp Business, confirmările și remindere-le automate merg prin integrarea oficială a platformei, nu printr-un tool neoficial și fragil care se poate strica oricând. Inbox-ul de e-mail e gestionat tot de AI: mesajele sunt clasificate automat (cereri de ofertă, programări, reclamații, spam), iar un draft de răspuns e generat pe baza contextului conversației. Când un caz depășește ce poate rezolva AI-ul, transferul către operatorul uman vine cu tot contextul conversației atașat, ca să nu fie nevoie ca clientul să repete de la capăt ce a spus deja.

Plan	Include	Minute/lună
Starter	Telefon SAU WhatsApp (un singur canal), programare automată	200
Business	Telefon + WhatsApp + E-mail, escaladare automată, jurnal complet	600
Enterprise	Canale nelimitate, integrare CRM/ERP existent, SLA contractual	negociat

Tehnologii: voce AI (Vapi), WhatsApp Business API, Gmail API, n8n, Make.

3.3.2. Automatizări Gmail & WhatsApp

Automatizează procesele repetitive din inbox-ul unei firme: răspunsuri la cereri de ofertă, urmărirea e-mailurilor rămase fără răspuns, sincronizare de calendar. E-mailurile primesc automat etichetă și rutare pe categorii, cu un draft de răspuns sugerat, iar programările pe WhatsApp primesc confirmare și remindere automate cu 24 de ore înainte. Google Calendar și CRM-ul clientului sunt sincronizate bidirecțional, iar acolo unde firma are deja alte unelte în uz (Slack, Notion, Sheets, Drive), le conectăm prin n8n, Make sau Zapier, ca să nu mai fie nevoie de copiat date manual dintr-un loc în altul.

Tehnologii: Gmail API, WhatsApp Business API, Google Calendar API, n8n, Make, Zapier.

3.3.3. Site-uri web

Dezvoltare de site-uri de prezentare, landing page-uri și portaluri client pentru firme B2B, orientate pe conversie și SEO tehnic, fără platforme de tip WordPress sau template-uri.

Pachet	Ce include	Livrare
Landing page	1–3 pagini, un singur obiectiv de conversie, SEO de bază	5 zile
Prezentare	până la 7 pagini, SEO tehnic complet, CMS pentru editare	7–10 zile
Avansat	7+ pagini, portal client cu autentificare, integrări API	la stabilit

Tehnologii: React, TypeScript, Vite, Tailwind CSS, Cloudflare Pages & Functions.

3.4. Proiect în dezvoltare: platformă SaaS pentru service-uri auto

În fază de concept și dezvoltare inițială, acest proiect reutilizează arhitectura deja validată în producție la VanaGreen (aceeași structură de module, aceeași abordare multi-tenant), aplicată unui domeniu diferit: service-uri auto, spălătorii, vulcanizări, tinichigerie-vopsitorie și dezmembrări.

Obiectivul e să înlocuiască fluxul actual, bazat pe deviz trimis pe WhatsApp și fișă de lucru pe hârtie, cu un flux digital complet. Doi diferențiatori pe care jucătorii existenți din piața românească nu îi au încă îmi par cei mai importanți de urmărit. Primul e inspecția digitală a vehiculului (DVI): un checklist pe tabletă pentru tehnician, cu cod de culoare pe severitate și fotografii sau video atașate, unde clientul aprobă separat fiecare element din deviz, nu un “da” generic pentru tot. Al doilea sunt indicatori reali pentru proprietarul afacerii — KPI-uri operaționale pe care service-urile auto din România pur și simplu nu le au azi disponibile automat.

Un singur tenant poate activa simultan mai multe tipuri de flux, de exemplu service, spălătorie și vulcanizare sub același acoperiș — situație frecventă pe piața locală, și unul dintre motivele pentru care arhitectura modulară de la VanaGreen se potrivește direct aici.

4. Workflow-ul unui proiect

Fluxul de lucru urmat pentru dezvoltarea și mentenanța fiecăruia dintre proiectele de mai sus este structurat în etape clar delimitate, indiferent de mărimea funcționalității livrate.

4.1. Discuția cu omul din spatele nevoii

Orice proiect pornește de la o discuție, nu de la o presupunere. Înainte să scriu o singură linie de cod sau să aleg vreo tehnologie, mă așez de vorbă, pe îndelete, cu omul care are nevoia reală: operatorul care completează hârtii, proprietarul afacerii, persoana care simte zilnic frustrarea unui proces greoi. Îl las pe el să-mi explice cum lucrează azi, unde pierde timp sau bani, ce a încercat deja și nu a funcționat, ce și-ar dori de fapt.

Nu construiesc plecând de la ce cred eu că ar trebui să existe sau de la ce “se poartă” în piață, ci de la nevoia reală a omului din fața mea. Discuția asta e primul pas, obligatoriu, înaintea oricărei linii de plan tehnic. Tot ce urmează mai jos există ca să răspundă la ce am aflat aici.

Un exemplu, fără date reale de client: un operator povestește că, la finalul fiecărei luni, un angajat petrece 3–8 ore calculând manual, dintr-un registru pe hârtie, un raport pe care trebuie să-l depună la autoritate. Din discuția asta reiese nevoia reală, care nu e “vreau o aplicație”, ci “vreau să nu mai pierd o zi de muncă în fiecare lună pe un calcul repetitiv”. Abia acum are sens să vorbim despre ce se construiește tehnic.

4.2. Planificare

Orice funcționalitate non-trivială pornește de la un plan scris explicit, înainte de a scrie cod. Un plan conține, la minimum: ce anume se schimbă și de ce (pornind direct din discuția de la pasul anterior), ce module sunt afectate, ce schimbări de schemă de date sunt necesare și ce compromisuri arhitecturale există, ce se câștigă și ce se pierde. Planul trece printr-o etapă de aprobare explicită înainte de a ajunge la implementare; nimic nu se implementează “din mers”, pe măsură ce apar idei noi în timpul scrisului de cod.

4.3. Implementare pe etape (“wave”)

Implementarea urmează o ordine fixă de dependențe, pentru a evita blocajele și integrările premature:

1. **Schema bazei de date și migrațiile.** Se stabilește întâi forma finală a datelor, pentru că o schimbare ulterioară de schemă costă mult mai mult decât una de cod aplicativ.
2. **Backend.** Rute, logică de business și validare, modul cu modul, fiecare izolat, fără dependențe încrucișate între module (vezi secțiunea 3.1).
3. **Frontend.** Componente și integrare cu un API deja funcțional și testat independent, ca un bug de UI să nu se confunde niciodată cu un bug de date.
4. **Verificare finală și documentare,** ultimul pas înainte de testarea propriu-zisă.

4.4. Testare

Testarea nu e un pas opțional la final, ci acoperă mai multe niveluri, fiecare cu alt scop. Testele unitare și de integrare (Vitest) verifică logica de business izolat, în special calculele și regulile de validare care n-au voie să greșească, cum ar fi calculul unui plafon legal. Testele end-to-end, cu Playwright, simulează un flux complet de utilizator în browser, de la login până la acțiunea finală, exact cum ar interacționa un client real cu aplicația.

Pe lângă acestea, verific automat și că interfața rămâne utilizabilă cu tastatura și cu cititoare

de ecran, nu doar vizual, și că fiecare pagină nouă respectă un prag minim de performanță (Lighthouse), nu doar că “merge”. Testarea manuală a fluxului complet rămâne ultimul filtru înainte de livrare — mai ales pentru cazurile limită, greu de automatizat complet oricât ai încerca.

4.5. Revizuire de cod și securitate

Fiecare schimbare trece printr-o etapă de revizuire de cod, urmărind trei categorii distincte: corectitudine (există un bug?), simplificare/reutilizare (există cod duplicat sau o soluție mai simplă?) și eficiență (există un cost de performanță nejustificat?).

Modificările care ating date personale, generarea documentelor legale, categorii reglementate legal sau regulile de reținere/ștergere a datelor trec printr-o etapă suplimentară de revizuire, dedicată exclusiv acestor aspecte, separată de revizuirea tehnică obișnuită, tocmai pentru că o greșeală aici are consecințe legale, nu doar tehnice.

4.6. Integrare continuă și livrare

Fiecare modificare trece printr-un pipeline automat înainte de a putea fi livrată: verificare de stil de cod (lint), verificare de tipuri, rulare a testelor automate, build complet, verificare de vulnerabilități în dependențe și scanare automată de secrete accidentale incluse în cod. Nicio modificare nu ajunge în producție dacă vreunul dintre acești pași eșuează.

Livrarea propriu-zisă este automatizată și diferă în funcție de tipul de produs: platformele SaaS rulează în containere Docker pe server propriu (Hetzner), cu reverse proxy Nginx și certificate SSL reînnoite automat; site-urile de prezentare se publică automat, direct din Git, către Cloudflare Pages, fără pas manual de deploy.

4.7. Documentare

La finalul fiecărei modificări semnificative actualizez un jurnal de decizii tehnice, cu patru elemente fixe: ce exista înainte, ce s-a schimbat, care e starea curentă și ce compromisuri au fost asumate. Ideea e ca orice decizie arhitecturală să poată fi reconstituită ulterior fără să depindă de memoria mea. Contează mai ales când mai multe produse independente (VanaGreen, VanaDash și celelalte) trebuie, ocazional, să comunice între ele: orice schimbare a acestui contract de integrare se documentează explicit, în ambele proiecte afectate, nu doar în cel unde a fost făcută schimbarea.

Concluzii

Cel mai mult mi-a rămas din perioada asta faptul că nu pot da vina pe altcineva pentru o decizie proastă. Când ești singurul dezvoltator, cel care vorbește cu omul din spatele nevoii e aceeași persoană care proiectează schema bazei de date, scrie backend-ul, scrie frontend-ul, testează, verifică securitatea și repară ce se strică duminică seara. Un compromis arhitectural asumat azi devine, garantat, problema mea de peste trei luni. Curricula facultății nu prea are cum să simuleze genul ăsta de presiune — și cred că e exact partea care lipsește cel mai des dintr-un proiect de laborator, oricât de bine ar fi construit tehnic.

Nu susțin că modul ăsta de lucru e ideal sau că merge scalat la nesfârșit; sunt lucruri pe care le-aș fi făcut altfel dacă aș fi avut de la cine să cer o a doua opinie tehnică înainte de a lua o decizie. Dar practica desfășurată în Iunie – August 2026 a fost, cel puțin, reală: produse care rulează în producție, cu clienți activi, nu un exercițiu de laborator scris ca să bifeze o cerință. Proiectul aflat acum în dezvoltare, platforma pentru service-uri auto, continuă pe același drum.